

ReBAC V2 — Recursive Authorization

Branch: ReBAC-V2 | **Builds on:** V1 Basic ReBAC

The Problem V1 Left Unsolved

In V1, a nurse assigned to a ward cannot access patients inside that ward — even though conceptually, the ward "contains" those patients and the nurse "owns" the ward.

The broken scenario:

```
Nurse Priya —[assigned_nurse]—> Ward-ICU
Ward-ICU —[contains]—> Patient101

checkPermission(Priya, Patient101, "view") → ✗ DENIED (V1)
```

The real-world requirement is:

"If you can view the ward, you can view everything contained in the ward."

This is a **recursive reachability** problem on a directed graph — the core algorithmic challenge of V2.

V2 Architecture

The Permission Rule Language (DSL Precursor)

```
resource patient {
  permission view =
    doctor_of
```

```

    OR contains -> view    ← "anyone who has 'view' on my parent
ward"
}

resource ward {
  permission view =
    assigned_nurse
    OR contains -> view    ← "anyone who has 'view' on my parent
department"
}

```

DFS Recursive Evaluation Algorithm

```

evaluateRecursive(userId, resource, permission):
  1. Find all parent resources via relation "contains"
  2. For each parent:
    subContext = { permission: "view" }
    result = checkPermission(userId, parent, subContext)
    if result == true → return ALLOW
  3. Return DENY

```

Graph traversal:

```

Patient101 ←[contains]– Ward-ICU ←[contains]– Dept-Surgery
                                     ↑
                                Nurse Priya –[assigned_nurse]–

```

DFS: Patient101 → Ward-ICU → Dept-Surgery → match! → ALLOW

Why DFS, Not BFS?

Algorithm	Property	Suitable For
DFS	Finds <i>any</i> valid path quickly, memory $O(\text{depth})$	Authorization (short-circuit on first match)
BFS	Finds <i>shortest</i> path, memory $O(\text{width})$	Shortest path queries, level-order discovery

DFS is correct here because we need to find *any* path that grants access, not the shortest one. However, DFS has a critical flaw: **unbounded recursion in cyclic graphs** — which V2 does not yet

handle.

Critical Review

✓ What V2 Gets Right

1. **DFS is the correct choice** for authorization reachability with short-circuit evaluation.
2. **Tuple-to-user set semantics** (`contains -> view`) correctly models hierarchical inheritance, matching Zanzibar's *tuple-to-user set rewrites*.
3. **Resource hierarchy** correctly models real-world hospital/corporate domain structures.

⚠ Critical Missing: Cycle Detection

This is a production bug.

If the resource graph contains a cycle:

```
A —[contains]—> B
B —[contains]—> A    (cycle!)
```

The DFS will loop infinitely → stack overflow → server crash.

V2 does not detect or handle cycles. This must be fixed in V2.1.

⚠ Critical Missing: Memoization

In a deep hierarchy with shared ancestors:

```
Patient1 → Ward-ICU → Dept-Surgery
Patient2 → Ward-ICU → Dept-Surgery ← same path traversed again!
```

Without memoization, the same subgraph is traversed repeatedly. For N patients sharing the same ward, complexity is $O(N \times \text{depth})$ instead of $O(\text{depth})$.

✗ Incorrect Assumption: DFS Termination

V2 assumes the resource graph is a DAG (Directed Acyclic Graph). In production databases, cycles **will** be introduced by incorrect data or concurrent writes. The engine must defend against this.

Comparison: How Zanzibar Handles Recursion

Google Zanzibar uses a **parallel BFS/DFS hybrid** with a depth limit and timeout, not pure DFS. It also:

- Evaluates multiple paths **concurrently**
- Uses **leopard caching** (distributed memoization)
- Enforces a **maximum tuple depth** (currently ~10 hops)

V2's serial DFS is correct but will not scale to millions of tuples without memoization and parallelism.

Missing Concepts vs Industry

Concept	V2	Zanzibar	SpiceDB
DFS traversal	✓	✓	✓
Cycle detection	✗	✓ (depth limit)	✓
Memoization	✗	✓ leopard cache	✓
Parallel evaluation	✗	✓	✓
Depth limit	✗	✓ (hardcoded)	✓ (configurable)
Timeout / deadline	✗	✓	✓

What The Next Version Must Fix

1. **Cycle detection** — visited set to prevent infinite loops
2. **Memoization** — `Map<cacheKey, boolean>` to avoid re-traversal
3. **Evaluation trace** — debugging which path granted/denied access
4. **Operator algebra** — AND / OR / NOT in permission rules

Summary

V2 correctly introduces recursive DFS traversal for hierarchical permission inheritance, directly mirroring Zanzibar's tuple-to-userset rewrite semantics. However, it is **not production-safe** without cycle detection and memoization. These are the two critical engineering gaps that V2.1 addresses.